Gmail

manoj kumar <manojkumarmk2608@gmail.com>

Case Study Round –Interview | Tredence Inc.

Zinia Lewis <zinia.lewis@tredence.com> Thu, Oct 15, 2026 at 12:08 PM
Cc: Soundarya M <soundarya.m@tredence.com>, Anusha V <anusha.v@tredence.com>, Priyanka Dwaraki <priyanka.dwaraki@tredence.com>, Lavanya M <lavanya.m@tredence.com>, Sandhya A <sandhya.a@tredence.com>

Dear Candidate,

As part of the interview process, we are sharing a case study for your review and completion. Kindly work on the attached case study and submit your response before attending the interview. This will enable the panel to evaluate your approach and discuss your solution in greater detail during the F2F round.

Deadline : 22nd Oct 2026

Case Study Submission _ AI Engineering Lead _ Tredence Inc. – Fill out form

Please note that the **second round of interviews will be based on this case study**. Candidates who clear the **MCQ assessment** will be required to present their case study solution to the interview panel.

If you are unable to complete the case study during the weekday, you may work on it **on Saturday after the MCQ assessment** and present your completed solution during the subsequent discussion.

We wish you the very best and look forward to your participation.

Please feel free to reach out to me or Soundarya from TA team.

Please find the case study, kindly submit the case study .

AQUA PLATFORM — TECHNICAL HIRING CHALLENGE

Role: Senior / Staff AI Platform Engineer Location: Bangalore, India Format: Take-home coding challenge (Python), followed by a live technical defense Time budget: 6–8 focused hours, to be completed within 5 calendar days of receipt Submission: A single GitHub repository (see Submission section below)

$B(!(B READ THIS FIRST
$B(!(B

This is not a challenge you can paste into an AI assistant and submit the result. We designed it that way on purpose.

There is no reference implementation of what we are asking for anywhere — not on GitHub, not in a paper, not in a blog post. The objective is a deliberate combination of ideas that does not exist as a ready-made recipe. To finish it you will have to derive things yourself, measure things yourself, and defend every decision in a live session where we will ask you to extend your own code in front of us.

You are welcome to use AI tools — we use them every day. But they will get you only part of the way here, and they will confidently get one specific thing wrong (we know which thing). Candidates who lean on them blindly will ship that bug and fail the defense. Candidates who understand the math will catch it. That is the point.

We are hiring for deep technical depth. This challenge measures exactly that.

$B(!(B ABOUT AQUA
$B(!(B

AQUA is our enterprise AI Agent Platform. We run agent inference at scale, which means we obsess over two things: the cost of every model we serve, and the correctness of the primitives no framework will write for you.

This challenge lives at that intersection. You will build a neural network that prunes itself while it trains, and you must prove it actually got cheaper — without overstating the result.

$B(!(B THE CHALLENGE: "THE SELF-PRUNING NETWORK" $B(!(B

You will build, from scratch in pure Python and NumPy, a small neural network library and use it to train a classifier that learns its own sparse structure during training — pruning away its least useful connections under a hard compute budget, while reasoning explicitly about the gradient consequences of doing so.

This is a real problem for us. On AQUA, a model that is four times smaller but 99% as accurate is often the difference between a feature shipping and not. The hard part is not "set small weights to zero." The hard part is doing it correctly: keeping your autodiff honest while the graph itself is changing underneath you, and proving the result is both sparse and accurate.

RESTRICTIONS (core implementation)

The following are not permitted for the core of your solution: PyTorch, TensorFlow, JAX, Keras, HuggingFace, or any autodiff, pruning, or neural-network library. Scikit-learn may be used only to load a standard dataset — not for training, models, or gradients — and you must state this explicitly in your README.

The autodiff engine, backward pass, pruning mechanism, optimizer, and training loop must be your own work.

PART 1 — A REVERSE-MODE AUTODIFF ENGINE THAT SURVIVES A CHANGING GRAPH (Mandatory)

Build a minimal reverse-mode automatic differentiation engine with the following:

1. A tensor type that wraps a NumPy array, tracks parents in a computation graph, and stores gradients.
2. Differentiable operations, at minimum:
    - Element-wise addition, subtraction, multiplication, division
    - Matrix multiplication
    - Reductions: sum and mean
    - Non-linearities: ReLU, and at least one of tanh, sigmoid, or GELU
    - A numerically stable softmax combined with cross-entropy loss
3. A backward() method with correct topological ordering and gradient accumulation, including the broadcasting case. For example, (N, D) + (D,) must backpropagate correctly.
4. Gradient checking against finite-difference numerical gradients, with automated tests that pass for every operation you implemented.

CORRECTNESS REQUIREMENT (we will verify this specifically)

Your engine must remain correct when parts of the parameter tensors are masked out (frozen to zero) between training steps.

A masked weight must contribute zero to the forward pass and receive a well-defined gradient treatment. Your optimizer state (momentum, Adam moments) for a pruned-then-revived connection must not silently corrupt training.

Think carefully about what "the gradient of a masked weight" should be, and what your optimizer does with it. Most quick implementations get this subtly wrong. Write a test that demonstrates you got it right.

PART 2 — TRAIN A NETWORK WITH A REAL OPTIMIZER (Mandatory)

Using only your Part 1 engine:

1. Implement a small MLP as composable layers (e.g. Linear + activation).
2. Implement Adam from scratch. SGD with momentum is the minimum acceptable optimizer, but Adam is strongly preferred for Part 3.
3. Train on a small classification task of your choice (e.g. MNIST, Fashion-MNIST, or a synthetic dataset such as spirals) using your own mini-batched training loop. Report a learning curve (loss and accuracy over epochs).
4. Justify your weight initialization choices. Demonstrate stable training without NaNs.

PART 3 — MAKE IT PRUNE ITSELF (The core of the challenge)

Train the same network so that it progressively removes its own least-important connections during training and ends under a target sparsity, while staying accurate.

You must design and implement the following yourself:

1. An importance criterion

   Decide how a connection earns the right to survive. Magnitude-only pruning is the trivial baseline — we will ask why you can do better. Stronger answers use gradient or sensitivity information (for example, a saliency estimate such as |weight $B!_(B gradient|, a Taylor expansion of the loss change from removing the connection, or a movement-based criterion). You must derive why your criterion approximates "loss increase if removed."

2. A pruning schedule

   Prune gradually under a schedule you justify (for example, a cubic sparsity ramp). One-shot pruning at the end of training is the weaker answer; explain the trade-off if you choose it.

3. Honest masking

   A pruned connection must be genuinely removed from the forward computation (a real zero, not a near-zero weight). Your autodiff and optimizer must handle the mask correctly (see the Part 1 correctness requirement). Bonus credit for allowing regrowth — a pruned connection returning if it becomes important again — and for reasoning about the stability implications.

4. A hard budget

   Hit a target sparsity (for example, 90% of connections removed) and report what it cost you in accuracy.

PART 4 — PROVE IT (Required evidence)

A pruned model that claims to be cheap but is not actually cheaper is worthless to us. You must produce evidence, not assertions:

1. A sparsity-accuracy Pareto curve

   Sweep target sparsity levels (for example: 0%, 50%, 75%, 90%, 95%). Plot final accuracy versus achieved sparsity. Commit both the plot and the raw numbers to your repository.

2. A real cost measurement

   Demonstrate that pruning actually reduced work — for example, measured FLOPs, active-parameter count, or wall-clock inference time on the sparse model using a sparse-aware forward path you wrote — not a dense forward pass that multiplies by zeros. If you use the dense-times-zero shortcut, say so explicitly and explain why it does not count as a real speedup. We would rather you be honest than overstate the result.

3. A baseline comparison

   Compare your importance-based pruning against pure magnitude pruning at the same sparsity. Is yours actually better? By how much? Is the difference real or noise? Show your reasoning about variance across random seeds.

4. One falsifiable claim

    State one specific, quantitative claim your results support (for example: "At 90% sparsity, saliency pruning retains X% accuracy versus Y% for magnitude pruning, averaged across N seeds") and back it with committed numbers in your repository.

$B(!(B LIVE TECHNICAL DEFENSE (Mandatory for shortlisted candidates, approximately 45 minutes) $B(!(B

Shortlisted candidates will join a live session where we will:

- Ask you to derive a gradient on a whiteboard, including the masked-weight case
- Ask you to add a new operation or a new importance criterion to your own code, live
- Probe the Part 1 correctness requirement and ask you to explain exactly how your optimizer handles a revived connection
- Ask how you would ship this inside a multi-tenant inference service at thousands of requests per second

If you did not write and understand your submission, you will not survive this session. That is intentional, and it is the most heavily weighted signal in our evaluation.

$B(!(B GROUND RULES $B(!(B

- Language: Python 3.10 or newer. NumPy is permitted throughout. Matplotlib is permitted for plots.
- Reproducibility: Fix your random seeds. We must be able to clone your repository and reproduce your reported numbers.
- AI assistants: Permitted, but you own and must defend every line of code and every claim in your documentation.
- Version control: Do not squash your commit history into a single commit. We review how you work, not just the final snapshot.

$B(!(B WHAT TO SUBMIT $B(!(B

Submit a GitHub repository containing:

1. Source code, organized clearly (suggested layout: engine/, nn/, prune/, train/, tests/).
2. A README.md with:
    - Installation and run instructions (we prefer uv, or a requirements.txt / pyproject.toml)
    - One command to run gradient-check tests
    - One command to reproduce Part 2 training
    - One command to reproduce the Part 3 self-pruning run
    - One command to reproduce the Part 4 Pareto sweep
3. Tests, including at minimum:
    - The gradient-checking suite from Part 1
    - The masked-weight correctness test from Part 1
4. Committed plots and raw numbers for Part 4.
5. A DESIGN.md document answering the following:
    - Derive your importance criterion and explain why it approximates the loss change from removing a connection.
    - What does your engine compute as "the gradient of a masked weight," and why is that the right choice?
    - Where does your autodiff engine bottleneck, and how would you optimize it?
    - How would you serve a self-pruned model in a real multi-tenant inference service at scale?

SUBMISSION MECHANICS

- Push your solution to a GitHub repository and share the link with us.
- If the repository is private, add the GitHub usernames we provide as collaborators.
- Include your full commit history.

$B(!(B HOW WE SCORE (100 POINTS) $B(!(B

Part 1 — Autodiff correctness, broadcasting, masked-weight handling 25 points Part 2 — Training loop and Adam optimizer working cleanly 10 points Part 3 — Self-pruning mechanism and derived importance criterion 25 points Part 4 — Honest evidence: Pareto curve, cost, baseline, falsifiable claim 20 points Code quality, tests, and reproducibility 10 points DESIGN.md reasoning and live defense 10 points Exceptional depth (regrowth, theory, ablations) up to +15 bonus

A correct, honest, well-defended submission that achieves 80% sparsity beats a flashy one that fakes its numbers or fails the live defense. We are scoring understanding, not output volume.

$B(!(B A NOTE ON WHAT WE ARE LOOKING FOR $B(!(B

Most submissions to "build a neural net" challenges look impressive and are subtly wrong: masked gradients that corrupt momentum, "sparsity" that is really just small weights, Pareto curves that do not reproduce from a clean clone. We have reviewed many of them.

The candidates we hire are the ones who caught the bug everyone else shipped, measured honestly, and could explain every line under pressure.

Regards,

Zinia Lewis

Talent Acquisition

zinia.lewis@tredence.com

www.linkedin.com/in/zinia-lewis-52469738a

TREDENCE Beyond Possible

The information contained in this transmission including attachments, if any, may contain privileged and confidential information which may be protected by federal and state laws. It is intended only for the use of the person(s) named in this communication. If you are not the intended recipient, you are hereby notified that any review, dissemination, distribution, or duplication of this communication is strictly prohibited, and the original message and all of its copies if any, must be destroyed immediately. Unless otherwise agreed, information does not form a binding legal contract by Tredence.